Inkcast end-to-end walkthrough — 0.1.0 at b716af5e2976

Inkcast 0.1.0 (commit b716af5e2976d816c5db9bf95e4880c48bf4563d, main) run in development mode against the fake DEV and Medium APIs, driven by a real MCP client (the Python SDK, OAuth with PKCE) and Playwright (Chromium 141.0.7390.37, Playwright 1.56.0, Python 3.14.8) through `http://localhost:8787`. Run 2026-10-22T16:00:36Z → 2026-10-22T16:00:50Z. Every number in this document is read from results.json, integrity.jsonl and env.json; nothing is typed in. Generated credentials are masked everywhere; the password is never written.

> **Outcome: PASS.** 30 of 30 steps passed (15 MCP calls, 6 REST requests, 8 browser pages); 8 of 8 integrity checks passed.

1. What was measured

Measure	Value
Inkcast server (MCP initialize)	inkcast 0.1.0; 9 tools: inkcast_platforms, inkcast_connect, inkcast_disconnect, inkcast_destinations, inkcast_publish, inkcast_sync, inkcast_posts, inkcast_cancel, inkcast_create_token
MCP client	Walkthrough MCP client — dynamic registration, PKCE S256, redirect to http://localhost:53682/callback
OAuth token	scope “inkcast”, expires in 3600 s
Steps	30 (30 passed, 0 failed): 15 MCP, 6 REST, 8 browser
Pictures	8 screenshots of Inkcast's pages, 21 rendered panels of MCP and REST calls
Draft on DEV	“Publishing from an MCP client” → https://dev.to/ada/article-1 (post pst_1a75c24665e9)
Scheduled then cancelled	“A story for later this hour” for 2026-10-22T17:00:43+00:00 (post pst_d9069b45936e)
Personal access token	ink_pat_16fbc3ed9546... (masked; scope stories:sync)
Git sync	github.com/acme/blog:posts/hello-from-a-repository.md → create published https://dev.to/ada/article-2
Ledger entries	1
Articles on the fake DEV at the end	2: “Publishing from an MCP client” (draft); “Hello from a repository” (published)
Integrity checks	8 of 8 passed
Run	2026-10-22T16:00:36Z → 2026-10-22T16:00:50Z

2. The journey, step by step

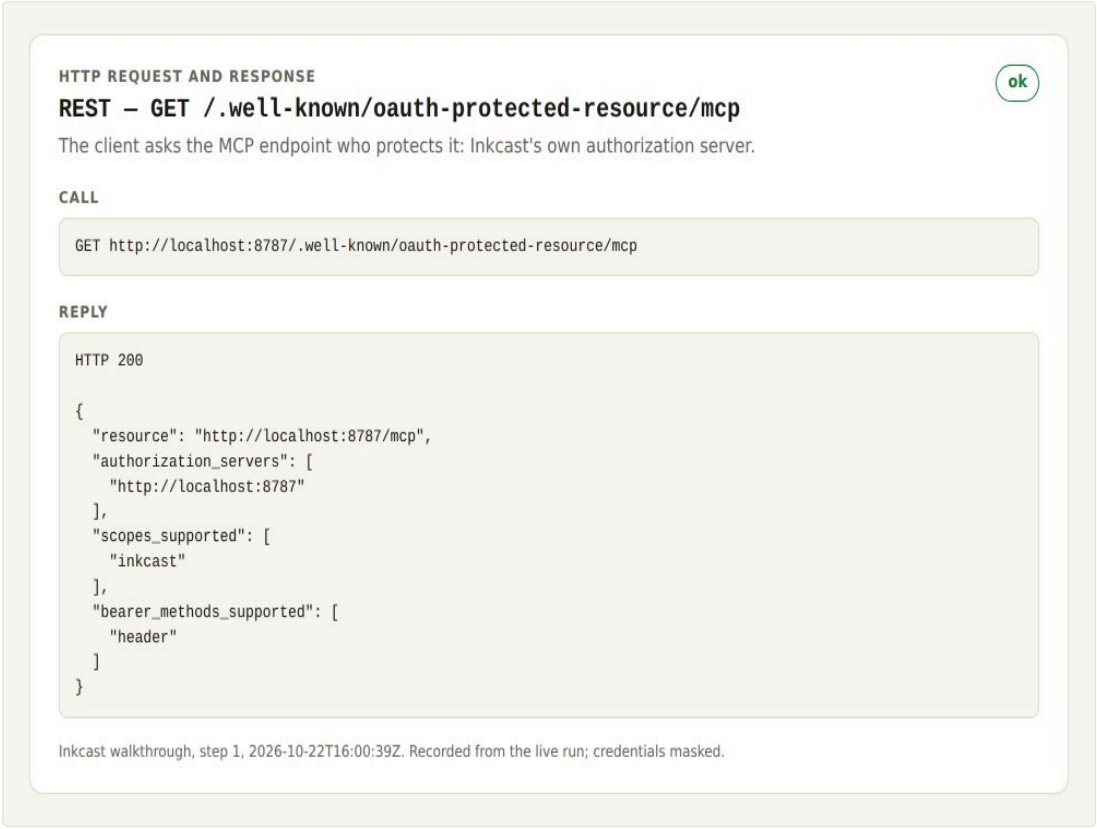
Each step is pictured as it happened: a screenshot of the Inkcast page the person saw, or a panel rendering the MCP tool call or REST request the client made and the reply it got. The line under each picture is what the step checked; a check that fails fails the step. A page with a JavaScript error, an HTTP error status or a visible error is refused rather than pictured as a success.

1. Add Inkcast to an MCP client

Step 1 — REST — GET /.well-known/oauth-protected-resource/mcp (rest, 2026-10-22T16:00:39Z)

The client asks the MCP endpoint who protects it: Inkcast's own authorization server.

✓ HTTP 200; the resource names its authorization server



01-rest-get-well-known-oauth-protected-resource-mcp.png — HTTP 200; the resource names its authorization server

Step 2 — REST — GET /.well-known/oauth-authorization-server (rest, 2026-10-22T16:00:39Z)

Discovery: where to register, authorize and get tokens; PKCE S256 is required.

✓ HTTP 200; metadata lists registration, authorization and token endpoints with S256

HTTP REQUEST AND RESPONSE

ok

REST — GET /.well-known/oauth-authorization-server

Discovery: where to register, authorize and get tokens; PKCE S256 is required.

CALL

```
GET http://localhost:8787/.well-known/oauth-authorization-server
```

REPLY

```
HTTP 200

{
  "issuer": "http://localhost:8787",
  "authorization_endpoint": "http://localhost:8787/authorize",
  "token_endpoint": "http://localhost:8787/token",
  "registration_endpoint": "http://localhost:8787/register",
  "scopes_supported": [
    "inkcast"
  ],
  "response_types_supported": [
    "code"
  ],
  "grant_types_supported": [
    "authorization_code",
    "refresh_token"
  ],
  "token_endpoint_auth_methods_supported": [
    "client_secret_post",
    "client_secret_basic"
  ],
  "revocation_endpoint": "http://localhost:8787/revoke",
  "revocation_endpoint_auth_methods_supported": [
    "client_secret_post",
    "client_secret_basic"
  ],
  "code_challenge_methods_supported": [
    "S256"
  ]
}
```

Inkcast walkthrough, step 2, 2026-10-22T16:00:39Z. Recorded from the live run; credentials masked.

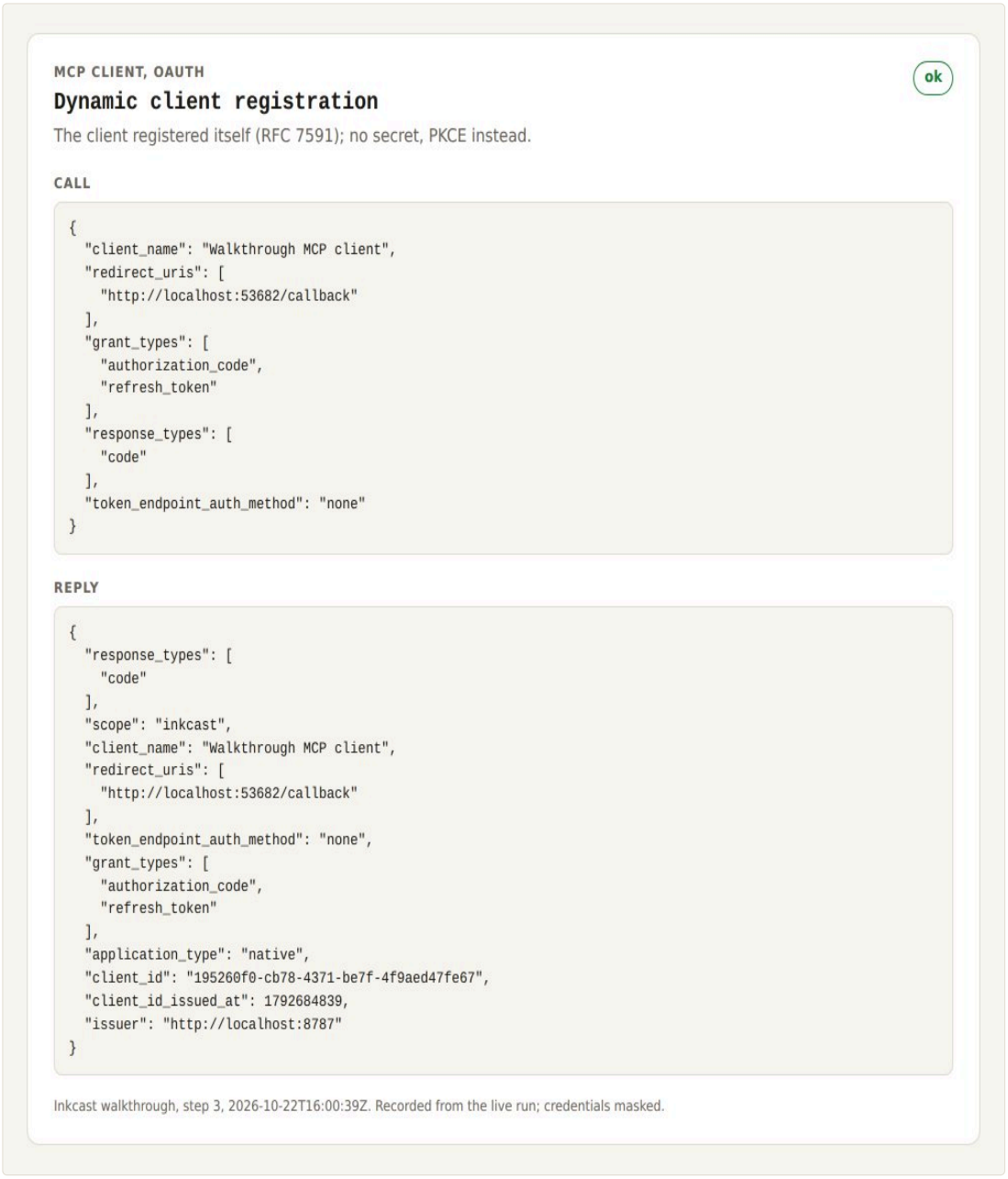
02-rest-get-well-known-oauth-authorization-server.png — HTTP 200; metadata lists registration, authorization and token endpoints with S256

Step 3 — dynamic client registration (mcp, 2026-10-22T16:00:39Z)

The client registered itself (RFC 7591); no secret, PKCE instead.

✓ registered as “Walkthrough MCP client” (client_id 195260f0-cb78-4371-be7f-4f9aed47fe67), redirect

http://localhost:53682/callback

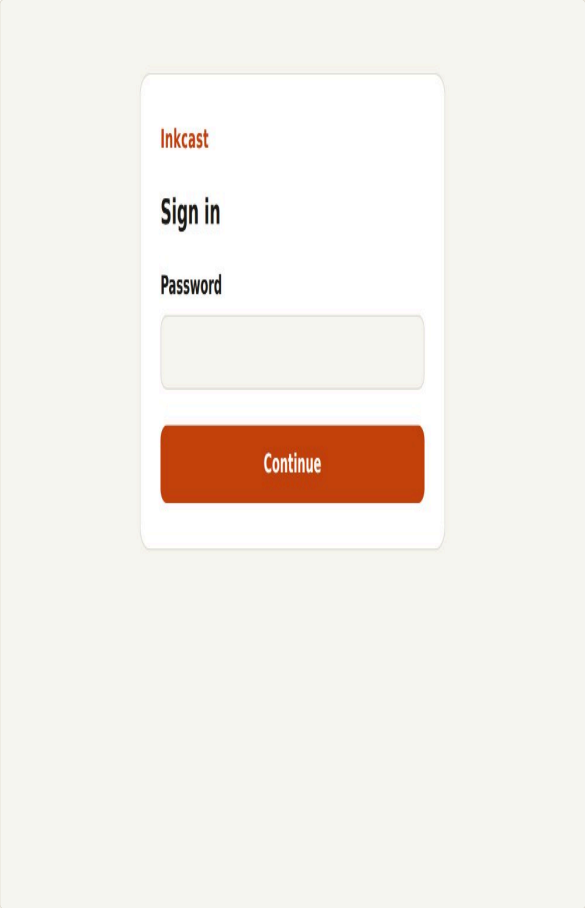


03-dynamic-client-registration.png — registered as “Walkthrough MCP client” (client_id 195260f0-cb78-4371-be7f-4f9aed47fe67), redirect http://localhost:53682/callback

Step 4 — sign-in page (browser, 2026-10-22T16:00:40Z)

The client opened the authorization URL in the browser; Inkcast asks the person to sign in.

✓ http://localhost:8787/auth/login?pending=MPLWJZo8zdb8Q2yzTCGhRBAwYeCFLQUq — Sign in; the sign-in form



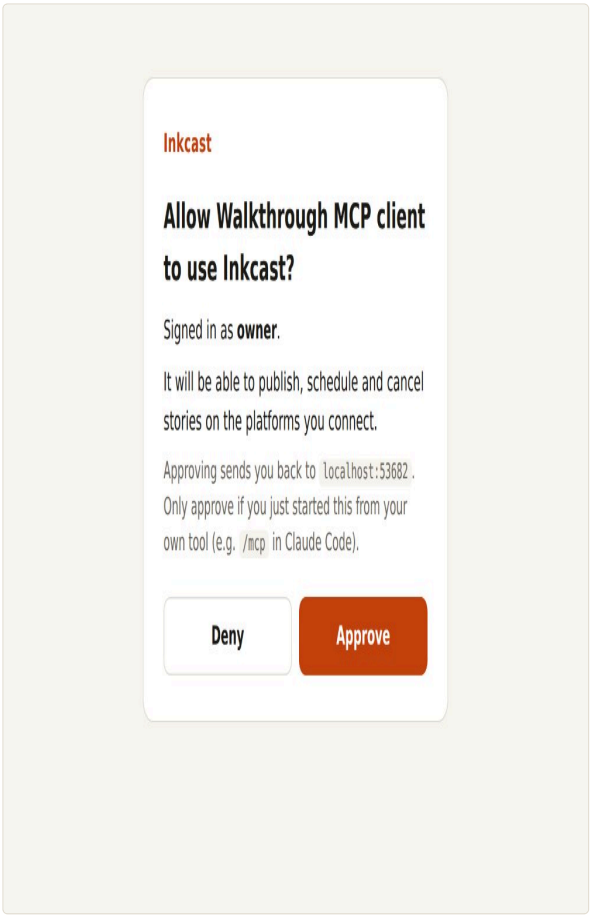
04-sign-in-page.png — http://localhost:8787/auth/login?pending=MPLWJZo8zdb8Q2yzTCGhRBAwYeCFLQUq — Sign in; the sign-in form

Step 5 — consent page (browser, 2026-10-22T16:00:41Z)

Signed in: Inkcast asks whether this client may use the account, naming the client.

✓ http://localhost:8787/auth/consent?pending=MPLWJZo8zdb8Q2yzTCGhRBAwYeCFLQUq — Allow Walkthrough MCP client to use

Inkcast?; consent names the client



05-consent-page.png — http://localhost:8787/auth/consent?pending=MPLWJZo8zdb8Q2yzTCGhRBAwYeCFLQUq — Allow Walkthrough MCP client to use Inkcast?; consent names the client

Step 6 — authorization code at the local callback (check, 2026-10-22T16:00:41Z)

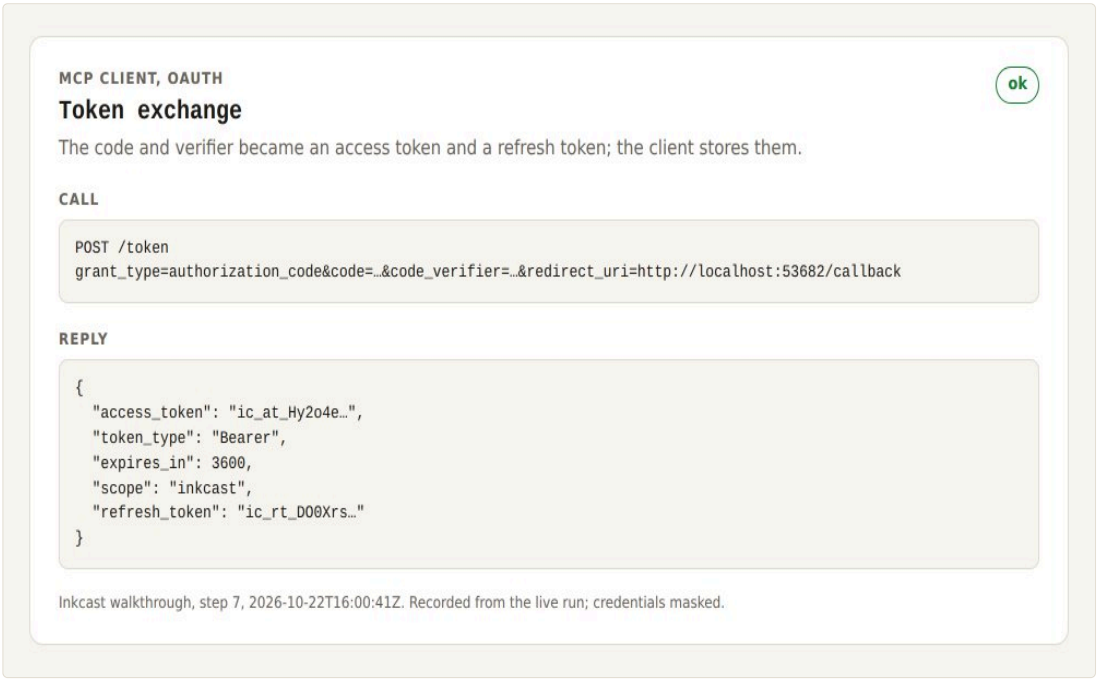
The browser was sent back to the client's local listener with a one-time code; the client exchanges it with its PKCE verifier.

✓ the redirect reached http://localhost:53682/callback with a code and state

Step 7 — token exchange (mcp, 2026-10-22T16:00:41Z)

The code and verifier became an access token and a refresh token; the client stores them.

✓ access token issued, expires in 3600 s, scope: inkcast

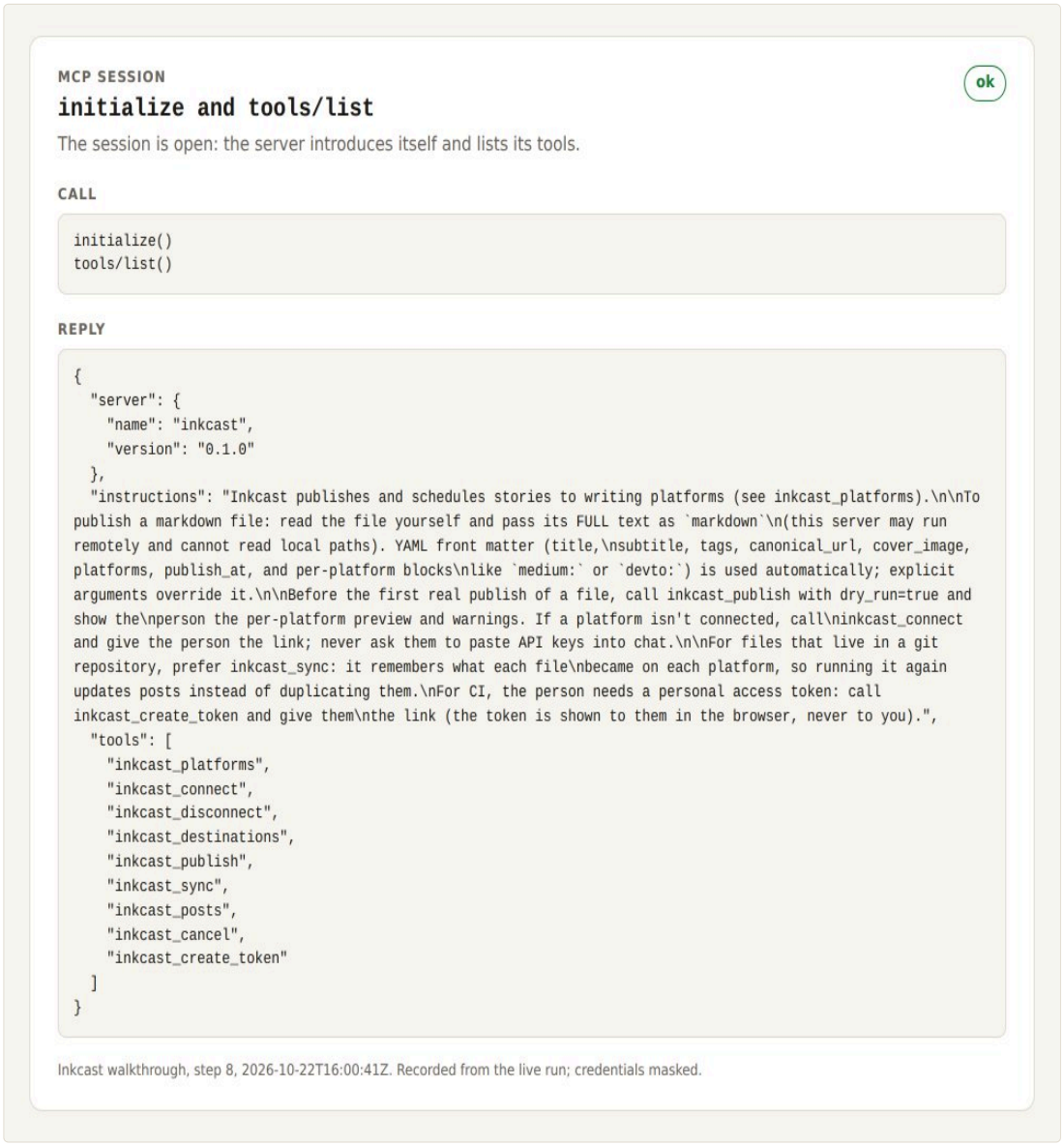


06-token-exchange.png — access token issued, expires in 3600 s, scope: inkcast

Step 8 — MCP initialize (mcp, 2026-10-22T16:00:41Z)

The session is open: the server introduces itself and lists its tools.

✓ server “inkcast” 0.1.0; 9 tools: inkcast_platforms, inkcast_connect, inkcast_disconnect, inkcast_destinations, inkcast_publish, inkcast_sync, inkcast_posts, inkcast_cancel, inkcast_create_token



07-initialize-and-tools-list.png — server “inkcast” 0.1.0; 9 tools: inkcast_platforms, inkcast_connect, inkcast_disconnect, inkcast_destinations, inkcast_publish, inkcast_sync, inkcast_posts, inkcast_cancel, inkcast_create_token

2. Platforms, connect DEV, destinations

Step 9 — MCP inkcast_platforms (mcp, 2026-10-22T16:00:41Z)

Nothing is connected yet.

✓ inkcast_platforms: DEV and Medium listed, both not connected



08-mcp-tool-call-inkcast-platforms.png — inkcast_platforms: DEV and Medium listed, both not connected

Step 10 — MCP inkcast_connect (mcp, 2026-10-22T16:00:41Z)

The assistant gets a one-time link for the person; the key never goes through chat.

✓ inkcast_connect: a one-time /connect/ link

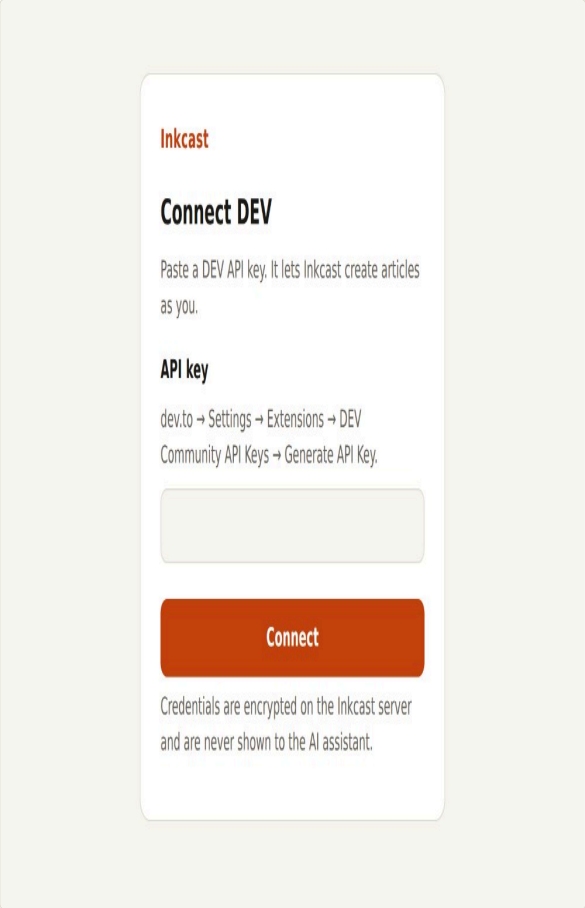


09-mcp-tool-call-inkcast-connect.png — inkcast_connect: a one-time /connect/ link

Step 11 — connect page (browser, 2026-10-22T16:00:42Z)

The person opens the link: a form for the DEV API key, with the help text.

✓ http://localhost:8787/connect/X2MpA_PiXKOoIOIJVWA_wHeNRWIhX2GmDC-ZyFe_Glw — Connect DEV; the DEV connect form



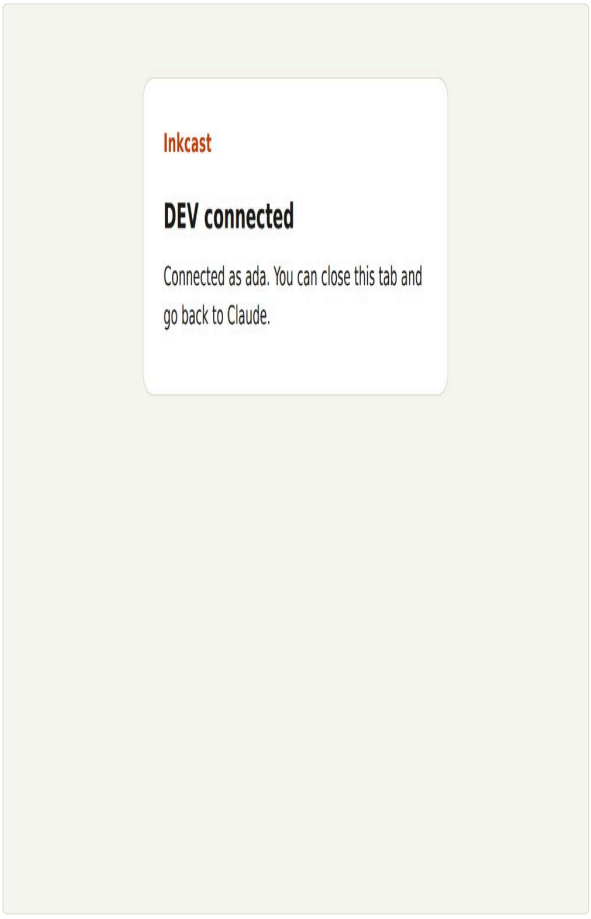
10-connect-page.png — http://localhost:8787/connect/X2MpA_PiXKOoIOIJVWA_wHeNRWIhX2GmDC-ZyFe_Glw — Connect DEV; the DEV connect form

Step 12 — dev connected (browser, 2026-10-22T16:00:43Z)

Inkcast verified the key with DEV and stored it; the link is used up.

✓ http://localhost:8787/connect/X2MpA_PiXKOoIOIJVWA_wHeNRWIhX2GmDC-ZyFe_Glw — DEV connected; the page says DEV

connected as ada



11-dev-connected.png — http://localhost:8787/connect/X2MpA_PiXKOoIOIJVWA_wHeNRWIhX2GmDC-ZyFe_Glw — DEV connected; the page says DEV connected as ada

Step 13 — MCP inkcast_platforms (mcp, 2026-10-22T16:00:43Z)

Back in the client: DEV is now connected.

✓ inkcast_platforms: DEV connected as ada



12-mcp-tool-call-inkcast-platforms.png — inkcast_platforms: DEV connected as ada

Step 14 — MCP inkcast_destinations (mcp, 2026-10-22T16:00:43Z)

Where else DEV can post: organizations, which DEV's API cannot list, so the reply says how to name one.

✓ inkcast_destinations: the reply explains organizations



13-mcp-tool-call-inkcast-destinations.png — inkcast_destinations: the reply explains organizations

3. Preview, publish a draft, list posts

Step 15 — MCP inkcast_publish (mcp, 2026-10-22T16:00:43Z)

A dry run first: DEV's version of the story, with any warnings, and nothing sent.

✓ inkcast_publish: a dry-run preview for DEV

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

MCP tool call — inkcast_publish

A dry run first: DEV's version of the story, with any warnings, and nothing sent.

CALL

```
inkcast_publish({
  "markdown": "<markdown: 17 lines, shown below>",
  "dry_run": true
})

markdown =
---
title: Publishing from an MCP client
subtitle: One markdown file, every platform, no copy and paste
tags: [writing, publishing, markdown, mcp]
canonical_url: https://blog.example.com/publishing-from-an-mcp-client
platforms: [devto]
---

# Publishing from an MCP client

You write the story once, in markdown, and your assistant publishes it where you asked.
Inkcast keeps the credentials; the assistant only ever sees the link.

## What happens

1. A dry run shows you each platform's version.
2. A draft lets you check it on the platform before it goes live.
```

REPLY

```
DRY RUN — nothing was sent.
“Publishing from an MCP client” — One markdown file, every platform, no copy and paste
[DEV]
  title: Publishing from an MCP client
  description: One markdown file, every platform, no copy and paste
  tags: writing, publishing, markdown, mcp
  canonical_url: https://blog.example.com/publishing-from-an-mcp-client
  status: published
  organization: (your profile)
```

Inkcast walkthrough, step 15, 2026-10-22T16:00:43Z. Recorded from the live run; credentials masked.

14-mcp-tool-call-inkcast-publish.png — inkcast_publish: a dry-run preview for DEV

Step 16 — MCP inkcast_publish (mcp, 2026-10-22T16:00:43Z)

Then the real thing, as a draft on DEV so it can be checked there first.

✓ inkcast_publish: a DEV draft with its URL and post id

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

MCP tool call — inkcast_publish

Then the real thing, as a draft on DEV so it can be checked there first.

CALL

```
inkcast_publish({
  "markdown": "<markdown: 17 lines, shown below>",
  "draft": true
})

markdown =
---
title: Publishing from an MCP client
subtitle: One markdown file, every platform, no copy and paste
tags: [writing, publishing, markdown, mcp]
canonical_url: https://blog.example.com/publishing-from-an-mcp-client
platforms: [devto]
---

# Publishing from an MCP client

You write the story once, in markdown, and your assistant publishes it where you asked.
Inkcast keeps the credentials; the assistant only ever sees the link.

## What happens

1. A dry run shows you each platform's version.
2. A draft lets you check it on the platform before it goes live.
```

REPLY

```
“Publishing from an MCP client” — One markdown file, every platform, no copy and paste
- devto: draft → https://dev.to/ada/article-1 (post `pst_1a75c24665e9`)
```

Inkcast walkthrough, step 16, 2026-10-22T16:00:43Z. Recorded from the live run; credentials masked.

15-mcp-tool-call-inkcast-publish.png — inkcast_publish: a DEV draft with its URL and post id

Step 17 — MCP inkcast_posts (mcp, 2026-10-22T16:00:43Z)

The post list shows the draft.

✓ inkcast_posts: the draft is listed

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

MCP tool call — inkcast_posts

The post list shows the draft.

CALL

```
inkcast_posts()
```

REPLY

```
Posts:
- [draft] 2026-10-22T16:00 devto: Publishing from an MCP client (post `pst_1a75c24665e9`) →
https://dev.to/ada/article-1
```

Inkcast walkthrough, step 17, 2026-10-22T16:00:43Z. Recorded from the live run; credentials masked.

16-mcp-tool-call-inkcast-posts.png — inkcast_posts: the draft is listed

4. Schedule a post, then cancel it

Step 18 — MCP inkcast_publish (mcp, 2026-10-22T16:00:43Z)

A second story scheduled an hour ahead (Inkcast refuses anything under two minutes away).

✓ inkcast_publish: scheduled for the requested time

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

MCP tool call — inkcast_publish

A second story scheduled an hour ahead (Inkcast refuses anything under two minutes away).

CALL

```
inkcast_publish({
  "markdown": "<markdown: 9 lines, shown below>",
  "publish_at": "2026-10-22T17:00:43+00:00"
})

markdown =
---
title: A story for later this hour
tags: [scheduling]
platforms: [devto]
---

# A story for later this hour

Scheduled from the MCP client, then cancelled before it went out.
```

REPLY

```
“A story for later this hour”
- devto: scheduled for 2026-10-22T17:00:43+00:00 (post `pst_d9069b45936e`)
```

Inkcast walkthrough, step 18, 2026-10-22T16:00:43Z. Recorded from the live run; credentials masked.

17-mcp-tool-call-inkcast-publish.png — inkcast_publish: scheduled for the requested time

Step 19 — MCP inkcast_posts (mcp, 2026-10-22T16:00:44Z)

Listed as scheduled, next to the draft.

✓ inkcast_posts: the scheduled post is listed

MODEL CONTEXT PROTOCOL, TOOLS/CALL

ok

MCP tool call — inkcast_posts

Listed as scheduled, next to the draft.

CALL

```
inkcast_posts()
```

REPLY

```
Posts:
- [draft] 2026-10-22T16:00 devto: Publishing from an MCP client (post `pst_1a75c24665e9`) →
https://dev.to/ada/article-1
- [scheduled] 2026-10-22T17:00 devto: A story for later this hour (post `pst_d9069b45936e`)
```

Inkcast walkthrough, step 19, 2026-10-22T16:00:43Z. Recorded from the live run; credentials masked.

18-mcp-tool-call-inkcast-posts.png — inkcast_posts: the scheduled post is listed

Step 20 — MCP inkcast_cancel (mcp, 2026-10-22T16:00:44Z)

Second thoughts: cancel it before it goes out.

✓ inkcast_cancel: the cancellation names the post



19-mcp-tool-call-inkcast-cancel.png — inkcast_cancel: the cancellation names the post

Step 21 — MCP inkcast_posts (mcp, 2026-10-22T16:00:44Z)

Now listed as cancelled; nothing reached DEV.

✓ inkcast_posts: the post is listed as cancelled



20-mcp-tool-call-inkcast-posts.png — inkcast_posts: the post is listed as cancelled

5. A personal access token for CI

Step 22 — MCP inkcast_create_token (mcp, 2026-10-22T16:00:44Z)

For CI the person needs a token the assistant never sees: another link to open.

✓ inkcast_create_token: a /tokens/new link with the name and scope

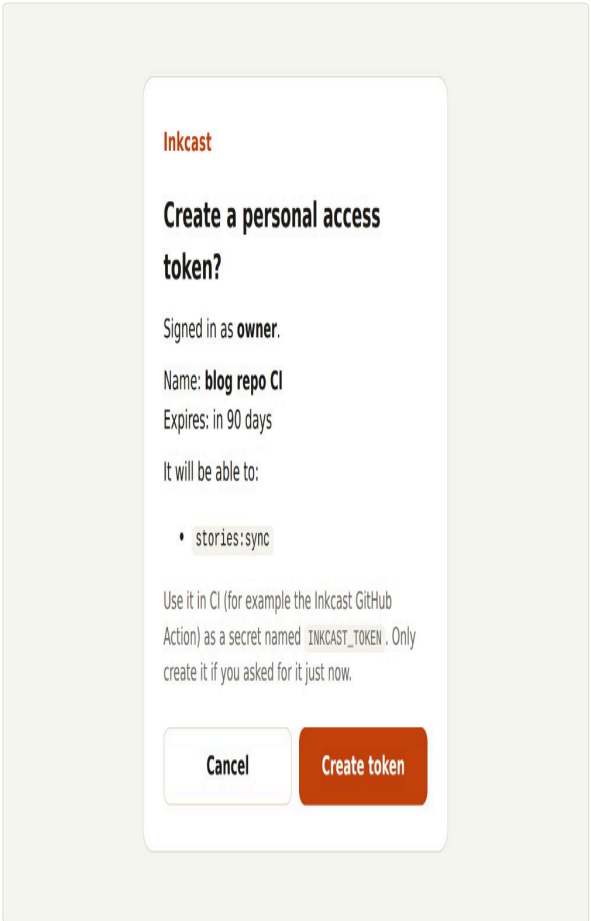


21-mcp-tool-call-inkcast-create-token.png — inkcast_create_token: a /tokens/new link with the name and scope

Step 23 — token confirm (browser, 2026-10-22T16:00:45Z)

After signing in again, the person confirms the token's name, scope and expiry.

✓ http://localhost:8787/auth/consent?pending=9BapPh82gIxtfIbOnT8ijM7I8CCPqugS — Create a personal access token?; the confirmation names the token and scope

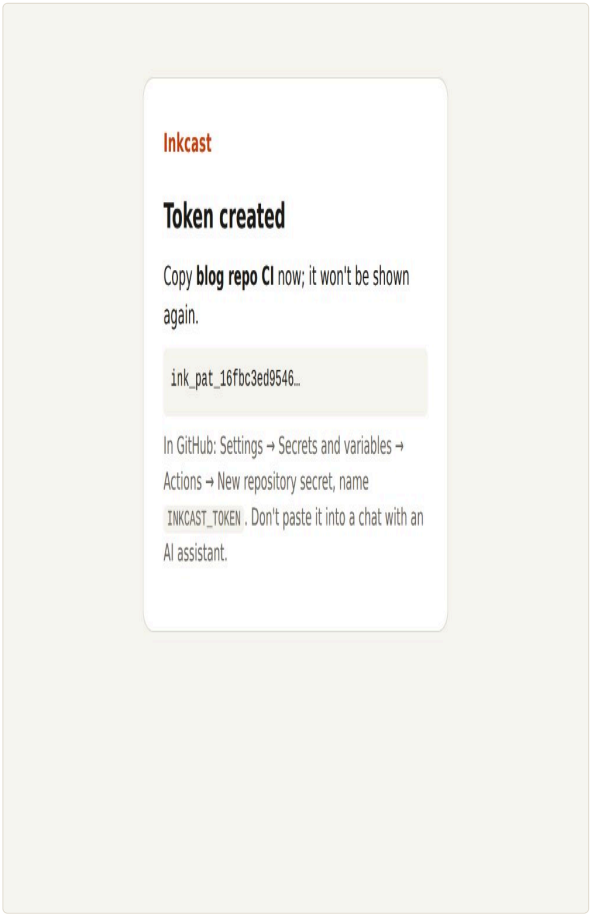


22-token-confirm.png — http://localhost:8787/auth/consent?pending=9BapPh82gIxtfIbOnT8ijM7I8CCPqugS — Create a personal access token?; the confirmation names the token and scope

Step 24 — token created (browser, 2026-10-22T16:00:45Z)

Shown once, to be stored as the CI secret INKCAST_TOKEN (masked here).

✓ http://localhost:8787/auth/consent — Token created; the token page, with the token masked



23-token-created.png — http://localhost:8787/auth/consent — Token created; the token page, with the token masked

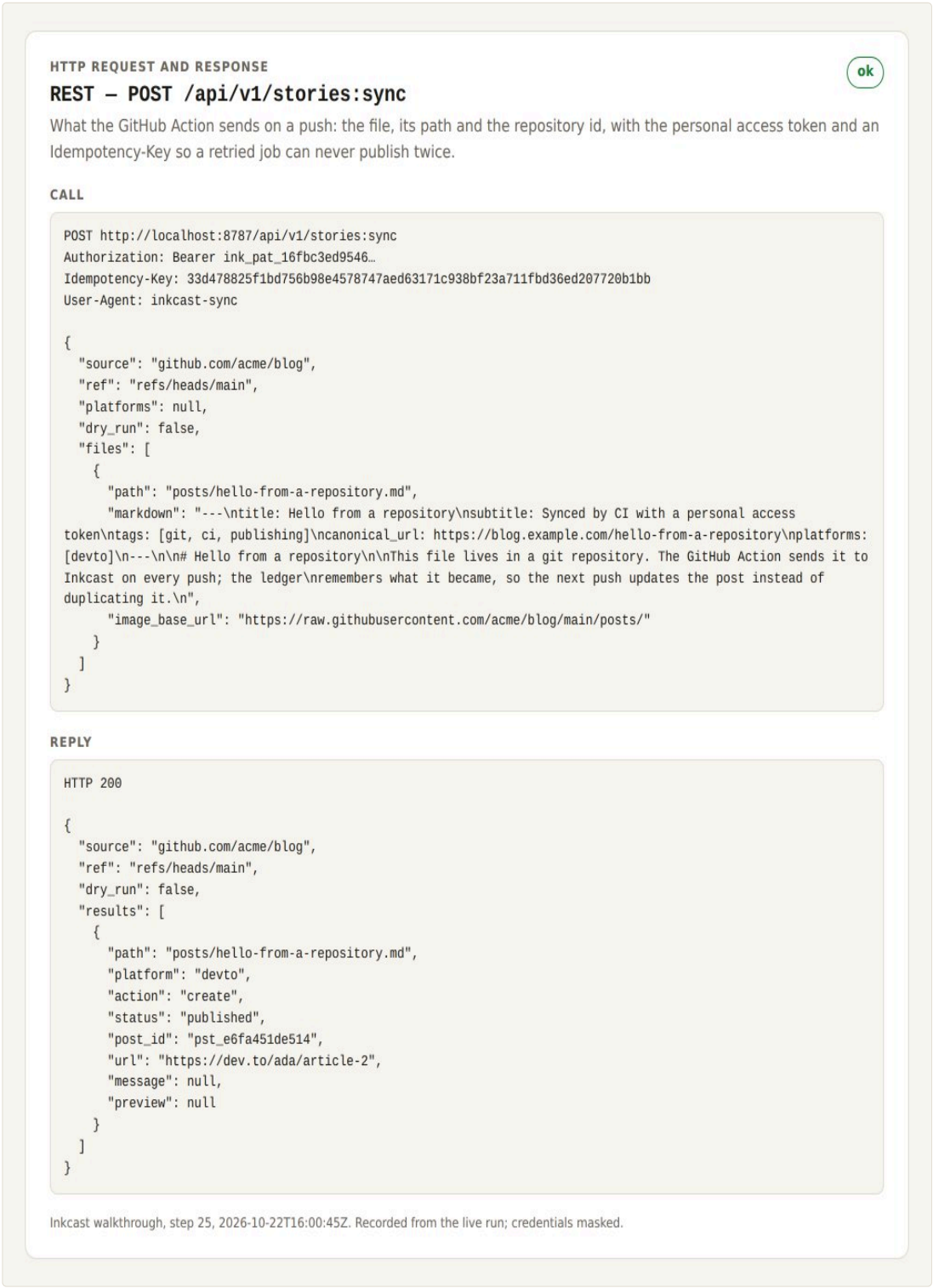
6. Git sync, as the GitHub Action does it

Step 25 — REST — POST /api/v1/stories:sync (rest, 2026-10-22T16:00:46Z)

What the GitHub Action sends on a push: the file, its path and the repository id, with the personal access token and an

Idempotency-Key so a retried job can never publish twice.

✓ HTTP 200; the file was created on DEV with a URL

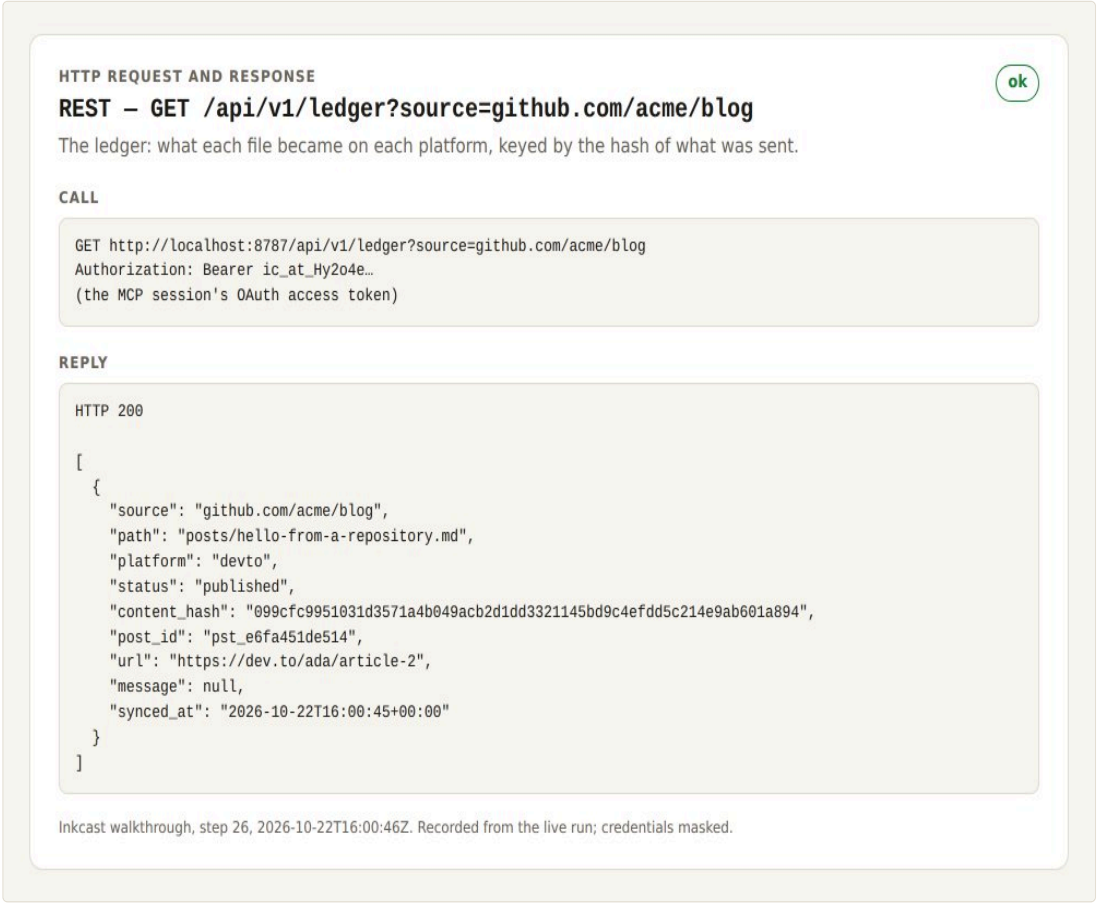


24-rest-post-api-v1-stories-sync.png — HTTP 200; the file was created on DEV with a URL

Step 26 — REST — GET /api/v1/ledger?source=github.com/acme/blog (rest, 2026-10-22T16:00:46Z)

The ledger: what each file became on each platform, keyed by the hash of what was sent.

✓ HTTP 200; the ledger has the file, published, with a content hash



25-rest-get-api-v1-ledger-source-github-com-acme-blog.png — HTTP 200; the ledger has the file, published, with a content hash

7. Insights

Step 27 — REST — GET /api/v1/insights/summary?days=7 (rest, 2026-10-22T16:00:47Z)

The person's own numbers, folded from domain events by the insights worker (settled after 2 poll(s)): posts by status, the ledger,

the sync outcomes, publish success and on-time rates.

✓ HTTP 200; the draft, the cancellation and the sync are all counted

HTTP REQUEST AND RESPONSE

ok

REST — GET /api/v1/insights/summary?days=7

The person's own numbers, folded from domain events by the insights worker (settled after 2 poll(s)): posts by status, the ledger, the sync outcomes, publish success and on-time rates.

CALL

```
GET http://localhost:8787/api/v1/insights/summary?days=7
Authorization: Bearer ic_at_Hy2o4e…
(the MCP session's OAuth access token)
```

REPLY

```
HTTP 200

{
  "days": 7,
  "scope": "user",
  "scope_id": "usr_e1a762af4ccb4c54",
  "posts_by_status": {
    "cancelled": {
      "devto": 1
    },
    "draft": {
      "devto": 1
    },
    "published": {
      "devto": 1
    }
  },
  "ledger_by_status": {
    "published": 1
  },
  "publish_success": [
    {
      "platform": "devto",
      "numerator": 2.0,
      "denominator": 2.0,
      "rate": 1.0
    }
  ],
  "on_time": [],
  "sync_by_action": {
    "create": 1
  },
  "sync_by_platform": {
    "devto": 1
  }
}
```

Inkcast walkthrough, step 27, 2026-10-22T16:00:47Z. Recorded from the live run; credentials masked.

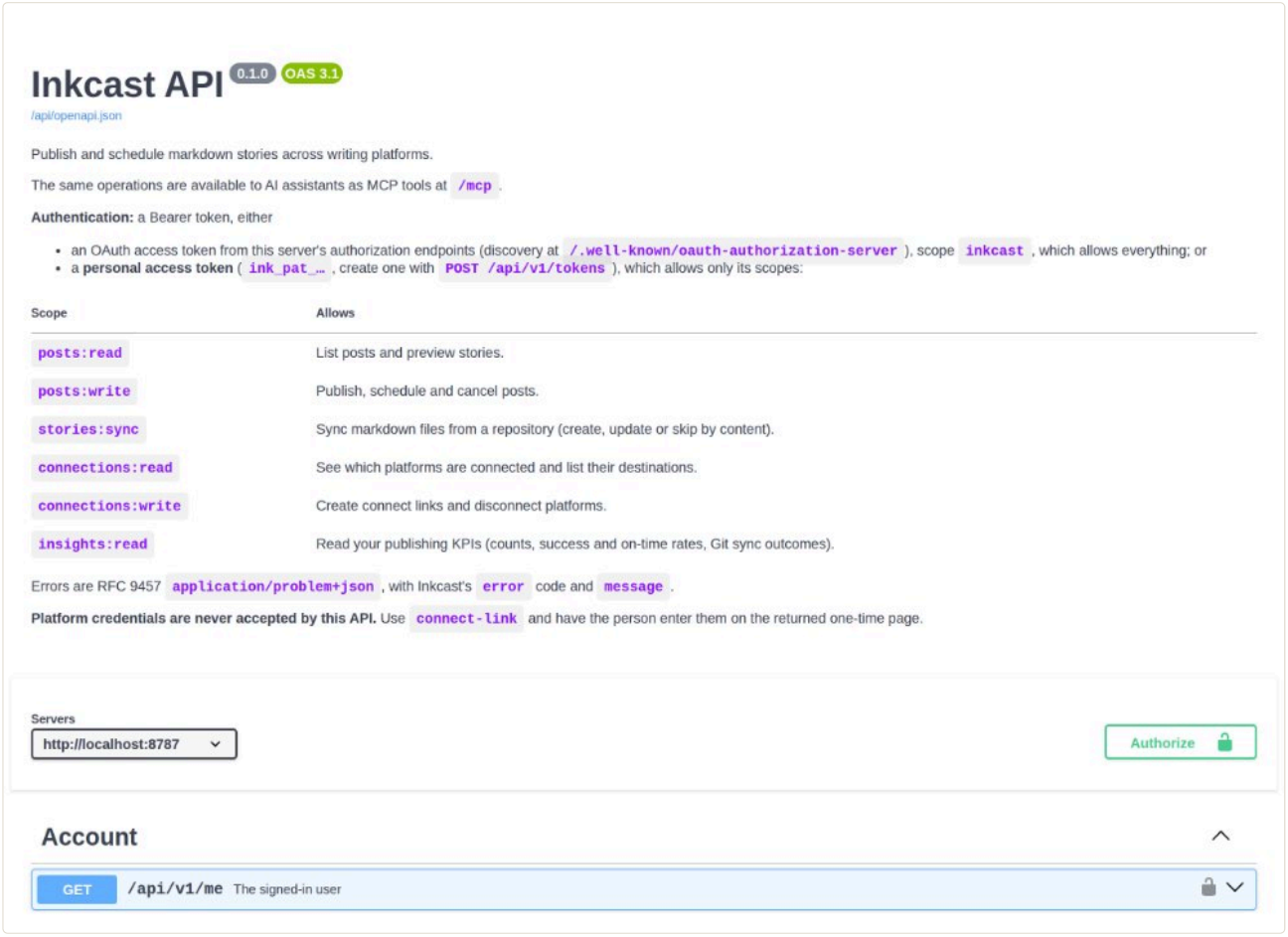
26-rest-get-api-v1-insights-summary-days-7.png — HTTP 200; the draft, the cancellation and the sync are all counted

8. The API reference

Step 28 — api swagger (browser, 2026-10-22T16:00:48Z)

The REST reference, served by every deployment with vendored assets (nothing from a CDN).

✓ http://localhost:8787/api — Inkcast API 0.1.0 OAS 3.1; Swagger UI lists the stories:sync route

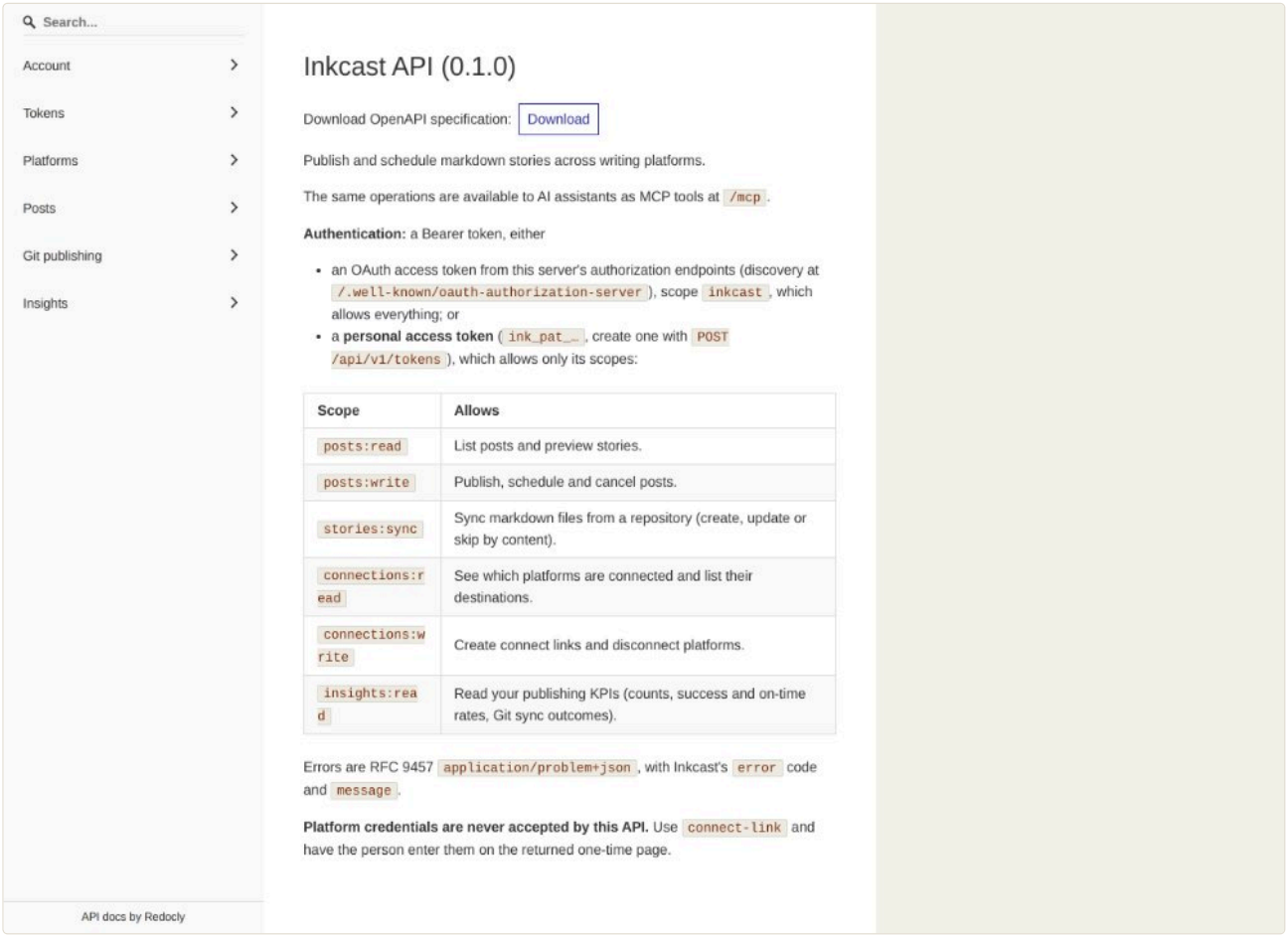


27-api-swagger.png — http://localhost:8787/api — Inkcast API 0.1.0 OAS 3.1; Swagger UI lists the stories:sync route

Step 29 — api redoc (browser, 2026-10-22T16:00:49Z)

The same contract in ReDoc.

✓ http://localhost:8787/api/redoc — Inkcast API (0.1.0); ReDoc renders the API title



28-api-redoc.png — http://localhost:8787/api/redoc — Inkcast API (0.1.0); ReDoc renders the API title

9. What the platform received

Step 30 — REST — GET /devto/api/articles/me/all (fake DEV) (rest, 2026-10-22T16:00:49Z)

Read back from the fake DEV, as the integrity check's witness: the draft and the synced file, and nothing for the cancelled

schedule.

✓ HTTP 200; two articles: the draft (unpublished) and the synced file (published)

HTTP REQUEST AND RESPONSE

ok

REST — GET /devto/api/articles/me/all (fake DEV)

Read back from the fake DEV, as the integrity check's witness: the draft and the synced file, and nothing for the cancelled schedule.

CALL

```
GET http://127.0.0.1:8999/devto/api/articles/me/all
api-key: devt…
Accept: application/vnd.forem.api-v1+json
```

REPLY

```
HTTP 200

[
  {
    "id": 1001,
    "title": "Publishing from an MCP client",
    "canonical_url": "https://blog.example.com/publishing-from-an-mcp-client",
    "url": "https://dev.to/ada/article-1",
    "published": false,
    "body_markdown": "You write the story once, in markdown, and your assistant publishes it where you
asked.\nInkcast keeps the credentials; the assistant only ever sees the link.\n\n## What happens\n\n1. A dry
run shows you each platform's version.\n2. A draft lets you check it on the platform before it goes live.\n"
  },
  {
    "id": 1002,
    "title": "Hello from a repository",
    "canonical_url": "https://blog.example.com/hello-from-a-repository",
    "url": "https://dev.to/ada/article-2",
    "published": true,
    "body_markdown": "This file lives in a git repository. The GitHub Action sends it to Inkcast on every push;
the ledger\nremembers what it became, so the next push updates the post instead of duplicating it.\n"
  }
]
```

Inkcast walkthrough, step 30, 2026-10-22T16:00:49Z. Recorded from the live run; credentials masked.

29-rest-get-devto-api-articles-me-all-fake-dev.png — HTTP 200; two articles: the draft (unpublished) and the synced file (published)

3. Integrity of what was left behind

Checked from the files alone, after the servers were stopped (integrity_check.py): the article the fake DEV holds is the story the client published (title, and the sha256 of the body re-rendered through the server's own DEV renderer); the ledger's content hash equals the hash the server computes for the synced file; the cancelled schedule never reached DEV; no credential appears in full in any output file.

Check	Result	Expected	Actual
DEV article: title of the draft	**pass**	`Publishing from an MCP client`	`Publishing from an MCP client`
DEV article: body of the draft (sha256)	**pass**	`8866e36398f42a875c1d8396b3c63493afd26a24325797ddddf0e3303fb72a3c`	`8866e36398f42a875c1d8396b3c63493afd26a24325797dddd`
DEV article: the draft is unpublished	**pass**	`false`	`false`
ledger: content_hash of the synced file	**pass**	`099cfc9951031d3571a4b049acb2d1dd3321145bd9c4efdd5c214e9ab601a894`	`099cfc9951031d3571a4b049acb2d1dd3321145bd9c4efdd5c2`
ledger: the synced file is published	**pass**	`published`	`published`
DEV article: body of the synced file (sha256)	**pass**	`60211cc4ac216304f2405bcdea991ef0e94612fd428a1ab426ab010d64d6bdc9`	`60211cc4ac216304f2405bcdea991ef0e94612fd428a1ab426a`
cancelled schedule: nothing on DEV	**pass**	`no article titled 'A story for later this hour'`	`[ "Publishing from an MCP client", "Hello from a re`
no credential in full in any output file	**pass**	`{}`	`{}`

4. Environment

Fact	Value
git_sha	`b716af5e2976d816c5db9bf95e4880c48bf4563d`
git_ref	`main`
inkcast_version	`0.1.0`
python	`3.14.8`
playwright	`1.56.0`
mcp	`2.3.0`
chromium	`141.0.7390.37`
os	`Linux-6.17.0-1022-azure-x86_64-with-glibc2.39`
ci	`39809980104`

Browser pages at 820×560 CSS px, device scale factor 2 (the API reference at 1280×900); panels at 1000 CSS px wide. The pictures in this document are downscaled copies; the originals are next to it.

5. Findings

No product finding was recorded for this run (no findings.json).

6. Files

File	Bytes	What
`01-rest-get-well-known-oauth-protected-resource-mcp.png`	126,323	picture
`02-rest-get-well-known-oauth-authorization-server.png`	216,855	picture
`03-dynamic-client-registration.png`	200,213	picture

04-sign-in-page.png	24,071	picture
05-consent-page.png	96,162	picture
06-token-exchange.png	111,389	picture
07-initialize-and-tools-list.png	272,009	picture
08-mcp-tool-call-inkcast-platforms.png	85,740	picture
09-mcp-tool-call-inkcast-connect.png	110,802	picture
10-connect-page.png	76,853	picture
11-dev-connected.png	34,862	picture
12-mcp-tool-call-inkcast-platforms.png	89,777	picture
13-mcp-tool-call-inkcast-destinations.png	102,297	picture
14-mcp-tool-call-inkcast-publish.png	268,371	picture
15-mcp-tool-call-inkcast-publish.png	222,006	picture
16-mcp-tool-call-inkcast-posts.png	91,039	picture
17-mcp-tool-call-inkcast-publish.png	151,886	picture
18-mcp-tool-call-inkcast-posts.png	109,140	picture
19-mcp-tool-call-inkcast-cancel.png	89,486	picture
20-mcp-tool-call-inkcast-posts.png	110,930	picture
21-mcp-tool-call-inkcast-create-token.png	135,841	picture
22-token-confirm.png	94,539	picture
23-token-created.png	72,907	picture
24-rest-post-api-v1-stories-sync.png	319,802	picture
25-rest-get-api-v1-ledger-source-github-com-acme-blog.png	181,528	picture
26-rest-get-api-v1-insights-summary-days-7.png	212,027	picture
27-api-swagger.png	285,380	picture
28-api-redoc.png	279,200	picture
29-rest-get-devto-api-articles-me-all-fake-dev.png	261,638	picture
env.json	288	data
fakes.log	0	process log
integrity.jsonl	1,669	data
results.json	33,526	data
server.log	16,221	process log